

Qué son?

Si ha revisado el código de algunas clases de Python como la de int, probablemente haya visto una sintaxis de `@palabra` arriba de algunos métodos:

Estos `@` se llaman `decorators` o `decoradores`, y son básicamente funciones que modifican y le dan funcionamiento extra a otras funciones (sean de una clase o no).

Por ejemplo, pueden usarse para ejecutar instrucciones *antes* o *después* de las instrucciones de la *función que decoran*.

También pueden usarse para revisar o modificar parámetros de la función.

Los `decoradores` en Python no son tan comunes, pero son muy útiles para automatizar tareas repetitivas que se deben hacer en distintas funciones.

Para crear un decorador, debemos seguir la siguiente sintaxis:

```
1 def decorator_name(func):  
2     def wrapper(parameters):  
3         # Logica extra  
4         func(parameters) # Llamada a la funcion decorada  
5         # Logica extra  
6  
7     return wrapper
```

[Copiar](#)

Por ejemplo, digamos que tenemos varias funciones que se pueden ejecutar solo por usuarios con el rol de administrador:

Copiar

```
1 class User:
2     role: str
3
4     def __init__(self, role):
5         self.role = role
6
7
8 def create_product(user, product_name):
9     if user.role != "Admin":
10         raise ValueError(
11             "You are not allowed to run this function. You are not an Admin"
12         )
13
14     # (lógica para crear producto...)
15     print(f"Product {product_name} created!")
16
17
18 def create_product_category(user, product_category_name):
19     if user.role != "Admin":
20         raise ValueError(
21             "You are not allowed to run this function. You are not an Admin"
22         )
23
24     # (lógica para crear categoria...)
25     print(f"Product Category {product_category_name} created!")
26
27
28 def modify_order(user, order_id):
29     if user.role != "Admin":
30         raise ValueError(
31             "You are not allowed to run this function. You are not an Admin"
32         )
33
34     # (lógica para modificar pedido...)
35     print(f"Order {order_id} modified!")
```

Copiar

```
1 my_user = User("Admin")
2 modify_order(my_user, 4)
```



Order 4 modified!

```
1 my_user = User("Customer")
2 modify_order(my_user, 4)
```

Copiar



ValueError: You are not allowed to run this function. You are not an admin

Tenemos lógica repetida que rompe con el principio `DRY`.

Una solución a esto sería convertir esa lógica de verificación en una función:

Copiar

```
1 class User:
2     role: str
3
4     def __init__(self, role):
5         self.role = role
6
7
8 def check_if_user_is_admin(user):
9     if user.role != "Admin":
10         raise ValueError(
11             "You are not allowed to run this function. You are not an admin."
12         )
13
14
15 def create_product(user, product_name):
16     check_if_user_is_admin(user)
17
18     # (lógica para crear producto...)
19     print(f"Product {product_name} created!")
20
21
22 def create_product_category(user, product_category_name):
23     check_if_user_is_admin(user)
24
25     # (lógica para crear categoría...)
26     print(f"Product Category {product_category_name} created!")
27
28
29 def modify_order(user, order_id):
30     check_if_user_is_admin(user)
31
32     # (lógica para modificar pedido...)
33     print(f"Order {order_id} modified!")
```

Sin embargo, tenemos que estarle pasando el user a la función, y también se vuelve algo repetitivo.

La solución más elegante y recomendada en estos casos donde hay que validar algo de manera repetitiva antes de la lógica de la función es crear un decorador:

Copiar

```
1 class User:
2     role: str
3
4     def __init__(self, role):
5         self.role = role
6
7
8 def admin_only(func):
9     def wrapper(user, *args):
10         if user.role != "Admin":
11             raise ValueError(
12                 "You are not allowed to run this function. Yo
13             )
14         func(user, args)
15
16     return wrapper
17
18
19 @admin_only
20 def create_product(user, product_name):
21     # (lógica para crear producto...)
22     print(f"Product {product_name} created!")
23
24
25 @admin_only
26 def create_product_category(user, product_category_name):
27     # (lógica para crear categoria...)
28     print(f"Product Category {product_category_name} created!")
29
30
31 @admin_only
32 def modify_order(user, order_id):
33     # (lógica para modificar pedido...)
34     print(f"Order {order_id} modified!")
```

Los decorators más usados

Abstract Method

Así es, ya usamos decoradores antes para crear métodos de clases abstractas en Los 4 Pilares de OOP!

Property

El decorator `@property` permite crear métodos que pueden ser llamados como si fueran atributos de la clase.

Estos son útiles para crear atributos *dinámicos*; con datos que se deben calcular a la hora de usarlos, y no de antemano - como un `age` calculado usando un `date_of_birth`.

Sin embargo, nunca deben ser usados en métodos que modifiquen información. Solo deben ser usados como getters.

Por ejemplo, veamos un student con dos notas y un promedio de ambas calculado en el constructor:

```
1 class Student:
2     spanish_score: int
3     english_score: int
4
5     def __init__(self, spanish_score, english_score):
6         self.spanish_score = spanish_score
7         self.english_score = english_score
8         self.average_score = (spanish_score + english_score)
9
10
11 student_ian = Student(80, 80)
12 print(f"Average score: {student_ian.average_score}")
13
14 student_ian.spanish_score = 50
15 print(f"Average score: {student_ian.average_score}")
```

[Copiar](#)

Average score: 80.0

Average score: 80.0

Como podemos observar, el `average` sigue siendo `80` a pesar de que su nota de español cambió.

Esto es incorrecto, así que deberíamos calcular el `average` de nuevo cada vez que cambia una nota, lo cual tampoco sería ideal.

O hacerlo un property que se calcule cada vez que se necesite.

```
1 class Student:
2     spanish_score: int
3     english_score: int
4
5     def __init__(self, spanish_score, english_score):
6         self.spanish_score = spanish_score
7         self.english_score = english_score
8
9     @property
10    def average_score(self):
11        return (self.spanish_score + self.english_score) / 2
12
13
14 student_ian = Student(80, 80)
15 print(f"Average score: {student_ian.average_score}")
16
17 student_ian.spanish_score = 50
18 print(f"Average score: {student_ian.average_score}")
```

Copiar

Average score: 80.0

Average score: 65.0

Veamos el ejemplo del `age` que mencionamos antes en acción:

```
1 from datetime import date
2
3
4 class User:
5     date_of_birth: date
6
7     def __init__(self, date_of_birth):
8         self.date_of_birth = date_of_birth
9
10    @property
11    def age(self):
12        # Debemos calcular la edad cada vez que la usemos
13        # ya que va a variar dependiendo de la fecha actual
14        today = date.today()
15        return (
16            today.year
17            - self.date_of_birth.year
18            - (
19                (today.month, today.day)
20                < (self.date_of_birth.month, self.date_of_birth.day)
21            )
22        )
23
24
25 my_user = User(date(1990, 1, 1))
26 print(f"Age: {my_user.age}")
```

Copiar



Age: 33

Class Method

El decorator `@classmethod` permite crear métodos en una clase que pueden ser llamados sin necesidad de instanciar la misma.

En otros lenguajes se les suele conocer como `static methods`.

Es decir que se pueden llamar directamente usando el nombre de la clase y el método sin tener que crear un objeto de la clase.

Esto podemos usarlo para funcionalidades que son parte de la clase, pero no necesariamente de un objeto específico de esa clase.

Por ejemplo, se suele usar para crear métodos que ayuden a crear instancias de la clase:

```
1 from datetime import datetime
2
3
4 class Person:
5     def __init__(self, first_name, last_name):
6         self.first_name = first_name
7         self.last_name = last_name
8
9
10 class User:
11     def __init__(self, email, password, person):
12         self.email = email
13         self.password = password
14         self.person = person
15
16 my_person = Person("Sarah", "Connor")
17 my_user = User("sconor@gmail.com", "321", my_person)
18
19 print("User: ", vars(my_user))
20 print("Person: ", vars(my_user.person))
```

Copiar

Copiar

```
1 from datetime import datetime
2
3
4 class Person:
5     def __init__(self, first_name, last_name):
6         self.first_name = first_name
7         self.last_name = last_name
8
9
10 class User:
11     def __init__(self, email, password, person):
12         self.email = email
13         self.password = password
14         self.person = person
15
16     @classmethod
17     def create_user(cls, first_name, last_name, email, password):
18         person = Person(first_name, last_name)
19         return User(email, password, person)
20
21
22 my_user = User.create_user("Sarah", "Connor", "sconor@gmail.com", "321")
23 print("User: ", vars(my_user))
24 print("Person: ", vars(my_user.person))
```



User: {'email': 'sconor@gmail.com', 'password': '321', 'person':
<main.Person object at 0x104a02710>}
Person: {'first_name': 'Sarah', 'last_name': 'Connor'}

En este caso, el `User` necesita también una instancia de `Person`.

Entonces tiene más sentido crear el `Person` y el `User` desde un método, y no desde el constructor del `User` (ya que el `Person` es una clase distinta).

Y este método no necesita un `User` previamente creado, así que tiene más sentido hacerlo un `classmethod`.

Por último, también podemos usarlos para funcionamientos externos a los Users, pero de igual forma relacionados a la clase, como intentos de login o registros:

pero de igual forma relacionados a la clase, como intentos de login o registros.

```
1 from datetime import datetime
2
3
4 users = [
5     {"email": "a@gmail.com", "password": "123"},
6 ]
7
8
9 class User:
10     email: str
11     password: str
12
13     def __init__(self, email, password):
14         self.email = email
15         self.password = password
16
17     @classmethod
18     def try_login(cls, email, password):
19         for user in users:
20             if user["email"] == email and user["password"] ==
21                 print("Login successful")
22                 return User(email, password)
23
24             raise ValueError("Wrong credentials")
25
26     @classmethod
27     def register(cls, email, password):
28         users.append({"email": email, "password": password})
29         return User(email, password)
30
31
32 my_new_user = User.register("c@gmail.com", "123")
33 User.try_login("c@gmail.com", "123")
```

[Copiar](#)